

LeaderKV: Improving Read Performance of KV Stores via Learned Index and Decoupled KV Table

Yi Wang[†], Jianan Yuan[†], Shangyu Wu[‡], Huan Liu[†], Jiaxian Chen[†], Chenlin Ma[†], Jianbin Qin[†]

[†] College of Computer Science and Software Engineering, Shenzhen University

[‡] Department of Computer Science, City University of Hong Kong

{yiwang, chenlin.ma, qinjianbin}@szu.edu.cn,

{yuanjianan2022, 2210273005, chenjiaxian2020}@email.szu.edu.cn,

shangyuwu2-c@my.cityu.edu.hk

Abstract—Log-structured merge-tree (LSM-tree) is a storage architecture widely used in key-value (KV) stores. To enhance the read efficiency of LSM-tree, recent works utilize the learned index to learn the mapping between keys and locations. However, in existing learned-index-aided KV stores, inefficient design of the learned index and disk access significantly impact the read performance. How to design a learned KV store to improve index efficiency and minimize disk access remains a critical problem.

This paper presents *LeaderKV*, a read-optimized LSM-tree-based KV store. *LeaderKV* employs decoupled KV tables (*DKTable*) and efficient learned indexes for data retrieval. *DKTables* are storage files in *LeaderKV* because they avoid reading irrelevant data in collaboration with learned indexes during queries. A learned index called *Leader* is proposed to accelerate data retrieval within *DKTable*. *Leader* is composed of precise models and approximate models. A redirect mechanism is designed to reduce the cost of mispredictions in *Leader*. We integrate *DKTable* and *Leader* into *LeaderKV* and demonstrate its effectiveness using a variety of datasets and workloads. Experimental results show that *LeaderKV* significantly improves the read performance compared to representative schemes.

Index Terms—Key-value store, LSM-tree, read performance, learned index, decoupled KV table

I. INTRODUCTION

Key-value (KV) databases are widely used in cloud computing, e-commerce, big data analysis, and artificial intelligence [1]. Examples are LevelDB [2] and BigTable [3] at Google, RocksDB [4] at Meta, Dynamo [5] at Amazon, Cassandra [6] and AsterixDB [7] at Apache. Among the architectures of KV stores, log-structured merge-tree (LSM-tree) [8] stands out for its remarkable write performance. LSM-tree transforms random writes into sequential writes, speeding up writing data to disk. However, LSM-tree gains excellent write performance at the expense of read performance. LSM-tree consists of multiple levels. Multiple files at different levels may be accessed during a point query, leading to read performance degradation.

A number of KV stores experience growing demands for reading. Yahoo reveals that the read proportion equals the write proportion in typical workloads in their KV stores [9]. The KV store benchmark, *Yahoo! Cloud Services Benchmark* (YCSB), shows a high proportion of read operations in core workloads [10]. In the workload analysis of UDB [11] and ZippyDB [12], which are databases derived from RocksDB,

the read operations account for the largest proportion [13]. Online Transaction Processing (OLTP), Artificial Intelligence (AI) model training, and Memcached show read-to-write ratios of 10:1, 9:1, and 7:1, respectively [14]. Most applications rely on efficient reading to provide high-quality services. Optimizing the read performance of KV stores has emerged as an urgent task.

Limitations of Existing Work: Many optimizations have been proposed to improve the read performance of LSM-tree-based KV stores [15], [16]. Some works utilize cache to accelerate lookups [17]–[19]. Although caching accelerates the retrieval of frequently accessed data, it introduces a non-negligible storage overhead as the data volume increases. Other works use filters to skip probing for keys if the filter indicates negatives [20]–[23]. However, filters may suffer inflated memory usage and high false-positive rates. Other works employ indexes like B+-Tree to retrieve data [24]–[26], but they increase storage overhead to keep indexes. The emerging learned index replaces traditional indexes with lightweight AI models to search data. It offers a promising solution for efficient data retrieval with low storage consumption [27]. Recent studies integrate learned indexes into LSM-tree [1], [28]. However, the read optimization of these works is constrained by the inefficient design of learned indexes and data access on disk.

Overview of This Work: This paper presents *LeaderKV*, a KV store that integrates learned index and decoupled KV table (*DKTable*) to accelerate data retrieval and minimize disk access. We introduce *DKTable* to replace Sorted String Tables (SSTable) to store KV pairs in *LeaderKV*. *DKTable* is partitioned into the key area and the value area. Keys are arranged orderly in the key area, and the associated values are stored in the value area. The decoupled structure of *DKTable* fits well with learned indexes to reduce the cost of disk access. Only a small amount of disk data is accessed during a query. To further accelerate data retrieval in *DKTables*, we introduce an efficient learned index called *Leader*. *Leader* consists of precise and approximate models and locates the target data via the hybrid-model structure. To reduce the cost of mispredictions, a redirect mechanism is designed in *Leader*. *DKTable* and *Leader* are integrated into *LeaderKV*. *LeaderKV* trains learned indexes in the background to minimize impacts

on foreground processes. The effectiveness of LeaderKV is evaluated by comparing representative schemes. Experimental results show that LeaderKV significantly improves read performance without compromising write performance.

Contributions. The main contributions of this paper are:

- To reduce the overhead of disk access during queries in read-intensive scenarios, we propose DKTable. DKTable collaborates with learned indexes to reduce data blocks loaded into memory.
- To accelerate lookup, we propose Leader, a learned index consisting of precise and approximate models. The hybrid-model structure of Leader enables efficient data retrieval within DKTables.
- We integrate DKTable and Leader into LeaderKV and conduct experiments to demonstrate its effectiveness.

The rest of this paper is organized as follows. Section II introduces the background and motivations. Section III presents the proposed DKTable and Leader. Section IV shows experimental results. Section V reviews the related work. Finally, Section VI concludes the paper and discusses future work.

II. BACKGROUND AND MOTIVATION

A. LSM-Tree-Based KV Stores

LSM-tree [8] is a write-efficient architecture widely employed in KV stores. For write operations, LSM-tree appends the data to *memtable* in memory. As memtable reaches its capacity limit, it is transformed into an immutable memtable (*imtable*) and later flushed to an SSTable on disk during minor compaction. LSM-tree organizes SSTables in a hierarchical structure. The newly generated SSTable is placed at a high level. Once the data volume at a level exceeds the capacity limit, a major compaction is triggered to merge the involved SSTables. During major compaction, the obsolete data is removed, and the most recent data is reserved in new SSTables at subsequent levels.

For read operations, LSM-tree first searches memtable and imtable. If the key is not found, LSM-tree searches SSTables from the highest level to the lowest level. Once the key is found in an SSTable, the value is returned. The detailed search process contains the following steps.

- **Seek Tables:** The SSTables that may contain the target key are identified.
- **Load IB/FB:** The index block and the filter block of the SSTable are loaded into memory.
- **Seek IB:** A binary search is performed within the index block to locate the data block.
- **Seek FB:** The filter block is queried to identify the existence of the key in the SSTable.
- **Load DB:** The located data block is loaded into memory if the filter indicates a positive.
- **Seek DB:** The loaded data block is searched to locate the target KV pair.

B. Learned Index

The learned index provides promising prospects for efficient data retrieval in database systems. The fundamental idea of

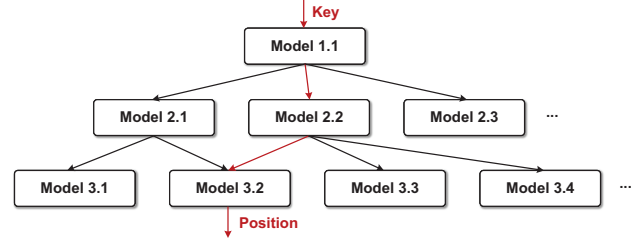


Fig. 1: Learned index structure.

the learned index is training machine learning (ML) models to build the mapping between keys and locations. A learned index trains complex models (e.g., neural network) or constructs a hierarchy of simple models (e.g., linear model). The classical learned index [27] employs ensemble learning and organizes models in a hierarchy, as shown in Figure 1. A query starts from the root node downwards. For a query key k , the model in layer L_i predicts the model to pick in layer L_{i+1} through the computation $F(k) \times N$, where N denotes the number of keys and F represents the cumulative distribution function (CDF) of keys. The CDF estimates the probability $p(x \leq k)$ of a key x less than or equal to the query key. Due to the training and inference overhead induced by complex models, most existing learned indexes employ linear models to fit the CDF. During queries, the index predicts the position (pos) of the key by computing $\lfloor pos = a \times k + b \rfloor$, where a and b are parameters learned from the training process. Benefiting from lightweight parameters (i.e., the slope and the intercept), learned indexes consume less memory than traditional indexes.

C. Motivation

Observed Performance Bottleneck Due to Inefficient Indexing. LSM-tree utilizes binary searches in index blocks and data blocks to retrieve KV pairs in SSTables. Bourbon [28] and TridentKV [1] utilize learned indexes for retrieval in SSTables. Ideally, with learned indexes, the target KV pair can be located through linear calculation without binary searches. However, Bourbon and TridentKV exhibit inefficiencies in their learned indexes. In Bourbon, the learned index comprises an array of linear models, which requires multiple binary searches to locate the target model. In TridentKV, the prediction granularity of learned indexes is the data block, which requires additional binary searches within the predicted data block.

A preliminary experiment is conducted to demonstrate the inefficiency of indexing. Since the source code of TridentKV is unavailable, we conduct experiments on its prototype, RocksDB, which also needs to load the data block and perform a binary search within it during queries. Both RocksDB and Bourbon preload 64 million records and perform 10 million random lookups. Figure 2 shows the normalized percentage of latency breakdown on four representative real-world traces. In Figure 2, labels “R” and “B” represent RocksDB and Bourbon, respectively. The proportion of Seek Key includes the proportion of Seek IB and Seek DB. From Figure 2, it

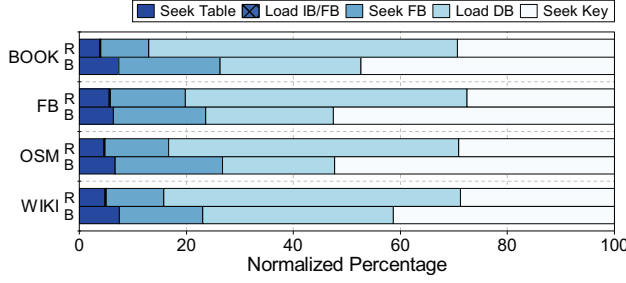


Fig. 2: Normalized percentage of random query latency breakdown.

can be observed that indexing (i.e., Seek Key) in RocksDB and Bourbon account for 28.70% and 48.34% on average, respectively. Therefore, there is a massive room for performance improvement in indexing.

Observed Performance Bottleneck Due to Disk Access.

In LSM-tree, the ideal scenario for retrieving a KV pair involves accessing only the target KV data. However, in existing KV stores, retrieval involves reading irrelevant data, causing read performance degradation. In LSM-tree, both index blocks and data blocks are loaded into memory to locate KV pairs. In Bourbon, numerous blocks are loaded in the case of mispredictions, leading to significant performance degradation [29]. In TridentKV, the entire predicted data block is loaded, causing overhead associated with disk access. From Figure 2, it can be seen that disk access (i.e., Load DB) in RocksDB and Bourbon account for 54.98% and 26.72% on average, respectively. Therefore, reducing disk access will bring a significant improvement in read performance.

Learned-Index-Aided Disk Access in LSM-tree. The learned index fits well with immutable SSTables to improve query efficiency. In learned indexes, it is promising to reduce binary searches by placing data at the predicted position. During queries, the predicted location is the exact one, minimizing the overhead of searching near the predicted location. Apart from faster lookup, reading fewer blocks is another effect of learned indexes. This helps reduce the cost of disk access, especially when a large proportion of data resides on disk like LSM-tree. Designing a fine-grained learned index can directly predict the location of KV pairs to reduce irrelevant data access. However, the design of such a learned index faces the following challenges. First, the value size of KV pairs in LSM-tree can vary significantly, complicating the model design to map keys to their locations. Second, in read-intensive workloads, point queries and range queries should be supported efficiently. Third, to balance lookup efficiency, training overhead, and space occupation, the learned index in LSM-tree should be carefully designed. Motivated by the observations and challenges, we design a collaborative method involving learned indexes and storage files to improve index efficiency and minimize disk access in LSM-tree.

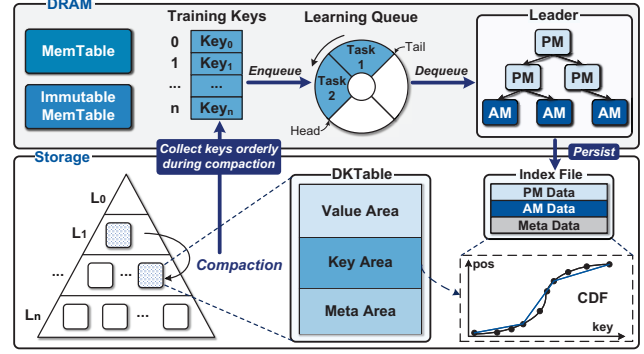


Fig. 3: The architecture of LeaderKV.

III. LEADERKV DESIGN

A. Overview

Figure 3 illustrates the architecture of the LeaderKV. LeaderKV is an LSM-tree-based KV store and replaces SSTables with DKTables. DKTable adopts table-level KV separation, which stores KV pairs in distinct areas. Specifically, the keys are stored in the key area in order, while the values are stored in the value area. The metadata of DKTable is stored in the meta area. DKTable significantly reduces the cost of disk access in collaboration with learned indexes. In DKTables, the CDF of keys stored in the key area can be modeled by learned indexes. A hybrid-model learned index called Leader is designed to improve the efficiency of retrieving data within DKTables. In Leader, the internal node utilizes a precise model (PM) to predict the node that should be fetched. The leaf node employs an approximate model (AM) to predict the area where the target data is located. During compaction in LeaderKV, keys in the newly generated DKTable are collected and submitted to the background learning queue for training Leader. After training, Leader is persisted in an index file on disk. The index file stores the parameters of precise models and approximate models, as well as the metadata of DKTable, which is necessary for model prediction.

B. DKTable: Table-Level KV Separation

The proposed DKTable aims to reduce irrelevant data access in collaboration with learned indexes during queries while keeping the spatial locality of keys and values. The main idea is table-level KV separation, which stores keys and values in different areas in DKTable.

Figure 4 presents the structure of DKTable. DKTable divides the file into the value area, key area, and meta area. DKTable splits each KV pair into key and value. Keys are stored in the key area in order, and values are stored in the value area in corresponding order to the keys. Both the value area and key area consist of blocks, which are the basic units for writing data to disk. The value's storage information (*vinfo*) is attached to the key to facilitate the retrieval of the value. The *vinfo* consists of the starting position of the value in DKTable (i.e., byte offset) and the size of the value. For example, *vinfo* #1

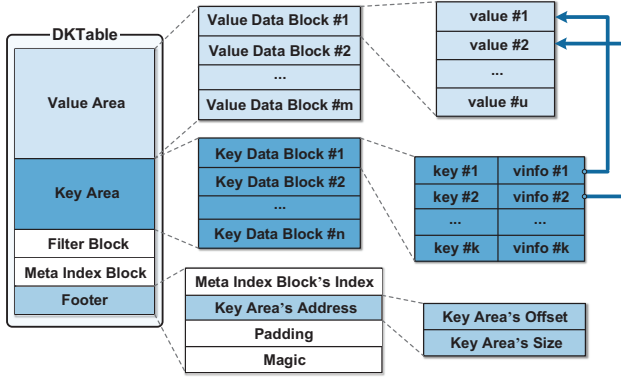


Fig. 4: The structure of DKTable.

consists of offset #1 and size #1. It is attached to key #1 to link value #1 in the value area. DKTable stores variable-sized values in the value area and fixed-sized key-vinfo pairs in the key area. This helps reduce the complexity of the design of learned indexes. The meta area contains a filter block, a meta index block, and a footer. The filter block stores the Bloom Filter. The meta index block stores the filter block's index. The footer stores the index of the key area (i.e., the offset and the size of the key area). It should also be noted that DKTable removes the index block. DKTable utilizes a learned index for data retrieval, with the learned index stored in a separate file. Each DKTable corresponds to a learned index file. As can be seen from experiments, the decoupled design of DKTable and the index file reduces the impacts of training and persistence of learned indexes on foreground processes.

To retrieve a KV pair in DKTable, the metadata stored in the meta area and the learned index stored in the index file are first loaded. Next, the learned index computes the logical position (pos) of the key in the key area (i.e., $pos = F(key) \times N$, where F is the CDF and N is the number of key-vinfo pairs in DKTable). In DKTable, both the key size (key_size) and the vinfo size ($vinfo_size$) are fixed. The fixed key size is a reasonable design since the key size in most datasets is fixed. The size of vinfo is 8 bytes, with the offset occupying 4 bytes and the value size occupying another 4 bytes. With the predicted logical position and the key area's offset ($offset$) read from the metadata, the byte offset of the key-vinfo pair in DKTable is computed as $offset + pos \times (key_size + vinfo_size)$. The Bloom filter is queried according to the predicted offset to identify whether the key exists in the DKTable. If the Bloom filter indicates a negative, the search process terminates. Otherwise, the key-vinfo pair is fetched at the byte offset. Lastly, if the fetched key matches the target key, the target value is read from the value area using vinfo. From the above process, it can be seen that the learned-index-aided DKTable can significantly reduce irrelevant data access during queries.

DKTables are constructed during minor compaction and major compaction in LeaderKV. In minor compaction, Lead-

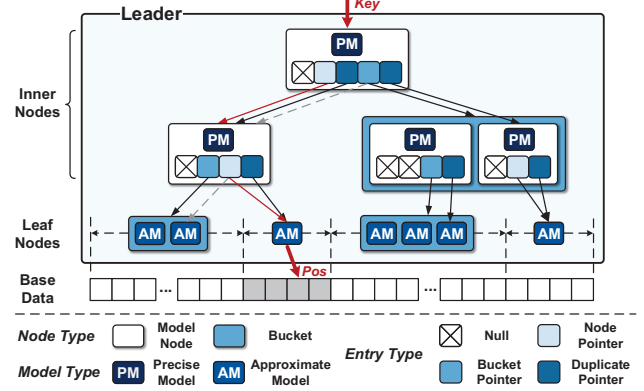


Fig. 5: The structure of Leader.

erKV writes the value of each KV pair to the new DKTable's value area and gains the vinfo. Then, key-vinfo pairs are written to the new DKTable's key area. In major compaction, LeaderKV employs a multiway iterator to iterate key-vinfo pairs in the key area of involved DKTables. Using the vinfo, the corresponding value is read from the old DKTable and written to the value area in the new DKTable. After the new vinfo is obtained, the new key-vinfo pair is written to the key area in the new DKTable, and the iterator moves to the next key-vinfo pair. During major compaction, the involved data blocks in the key area and the value area are cached, ensuring the efficiency of reading keys and values. After the key area and the value area are constructed, the metadata is written to the meta area in DKTable.

C. Leader: A Hybrid-Model Learned Index for DKTable

In the search process in DKTable, a fine-grained learned index whose prediction granularity is KV pairs can significantly reduce the cost of disk access. However, designing such an index requires careful consideration of query efficiency, space consumption, and training overhead. We present Leader, a hybrid-model learned index to improve query performance in DKTable without bringing significant overheads. Figure 5 shows its structure, which is set to a height of three layers. There are two types of nodes in Leader:

- **Model Node.** The model node is the basic unit in Leader. A model approximates the key distribution in the node. The model nodes are categorized into two types: the precise model node (PM node) at the internal layers and the approximate model node (AM node) at the leaf layer. Each PM node contains a precise model, an array of entries, and a bit vector of entry types. The model of PM node is trained before node construction. During construction, PM node utilizes the model to store child pointers at predicted entries. Each AM node is an approximate model consisting of the slope and the intercept. AM nodes are constructed using piece-wise linear regression (PLR) [30]. The gap between the predicted position and the exact position is within an error bound.

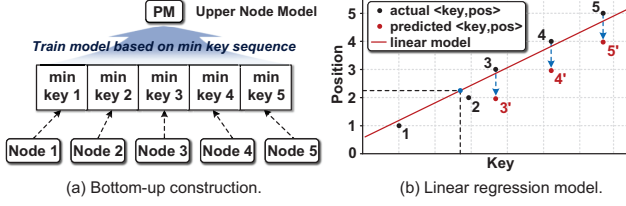


Fig. 6: An example to illustrate the redirect mechanism.

- **Bucket.** The bucket is a buffer that uses an array to store model nodes that are in conflict. Conflicts arise when multiple model nodes are predicted at the same position. For these conflict model nodes, the bucket stores them in ascending order based on their key range.

In PM nodes, the entries store different types of pointers, and the entry type indicates the type of pointer stored in the entry. There are four types of entries in PM nodes:

- **Node Pointer.** This type of entry stores a pointer containing the address of a model node.
- **Bucket Pointer.** This type of entry stores a pointer containing the address of a bucket.
- **Duplicate Pointer.** This type of entry stores the duplicate address of the child model node, which is indexed by the previous node pointer or bucket pointer. This type of pointer is discussed in the following.
- **Null.** This entry is an unused slot.

Leader is constructed from the bottom up. As Figure 6(a) shows, the model of PM node is trained based on the minimum keys extracted from the children (i.e., $Node_1$ to $Node_5$). First, the minimum key of each child (i.e., $min\ key_1$ to $min\ key_5$) is collected from $Node_1$ to $Node_5$, which forms an ordered minimum key sequence. Subsequently, the ordinary least squares (OLS) [31] regression is applied to this sequence to train the linear model of PM node.

To reduce the cost of mispredictions during queries, a redirect mechanism is proposed, including the design of duplicate pointers and redirect pointers. The duplicate pointer and the redirect pointer aim to re-navigate the predicted position to the correct position. In the case of mispredictions, the re-navigated model node is fetched directly using duplicate pointers or redirect pointers. If the key is not in the range of the redirect node, it indicates that the key does not exist in the index, and the search is terminated. This ensures that each PM node requires only one linear calculation and, at most, a redirection to find the child, thus reducing the cost of mispredictions.

To further explain the redirect mechanism, a simple example is provided in Figure 6(b), which presents the linear regression result in Figure 6(a). The black dots represent the minimum keys from $Node_1$ to $Node_5$, along with their actual positions (p_1 to p_5). From the regression results, it can be observed that $min\ key_1$ and $min\ key_2$ are predicted at p_1 and p_2 after rounding down, respectively, which match their actual positions. The $min\ key_3$, $min\ key_4$, and $min\ key_5$ are predicted at p_2 , p_3 , and p_4 after rounding down, respectively. Since $min\ key_2$ and $min\ key_3$

TABLE I: A case study: child pointers and redirect pointers in the entries of the root node in Figure 5.

#th Entry	Entry Type	Child Pointer	Redirect Pointer
1	Null	Null	Null
2	Node Pointer	PM node ₁	Null
3	Duplicate Pointer	PM node ₁	Null
4	Bucket Pointer	bucket	PM node ₁
5	Duplicate Pointer	PM node ₃	Null

key_3 are predicted at the same position (i.e., p_2), $Node_2$ and $Node_3$ are contained in the same bucket in construction. For $Node_1$, $Node_4$, and $Node_5$, their addresses are contained in the 1st, 4th, and 5th entries, respectively. Assume that a query key falls in the range greater than $min\ key_1$ and less than $min\ key_2$, much closer to $min\ key_2$, as shown in Figure 6 (b). This key should be searched in $Node_1$. However, from the regression results, the predicted position of the query key is p_2 after rounding down, resulting in the navigation to $Node_2$. To redirect the predicted child to $Node_1$, the parent node stores an additional redirect pointer pointing to $Node_1$ at the predicted entry. From the above process, it can be seen that when the key falls outside the range of the predicted child, the redirect child can be directly fetched using the redirect pointer.

The gray dashed lines in Figure 5 represent the redirect pointers. For entries whose types are node or bucket, they store both child pointers (i.e., node pointers and bucket pointers) and redirect pointers. The redirect pointer points to the redirect child node. Suppose $prep$ represents the child indexed by the previous node pointer or bucket pointer. If $prep$ is an AM node or PM node, the redirect pointer contains the address of $prep$. If $prep$ is a bucket, the redirect pointer contains the address of the last model node in $prep$. The duplicate pointer is a special type of the redirect pointer. For duplicate entries, they do not have child pointers initially because none of the minimum keys are predicted at these positions. Thus, duplicate entries store the redirect pointer as the child pointer. Based on the principle, we take the root node in Figure 5 as a case study, whose child pointers and redirect pointers are listed in Table I. Suppose PM nodes in the second layer are $PM\ node_1$, $PM\ node_2$, and $PM\ node_3$, from left to right. The minimum key of $PM\ node_1$ is predicted at the second entry, and there is no model node previous to $PM\ node_1$. Thus, the redirect pointers of the first and the second entry are null. The minimum keys of $PM\ node_2$ and $PM\ node_3$ are both predicted at the fourth entry. Therefore, the third entry stores the redirect pointer as the child pointer, pointing to $PM\ node_1$. The fourth entry also stores the redirect pointer, pointing to $PM\ node_1$. Lastly, the fifth entry stores the redirect pointer as the child pointer, pointing to the last model node in the bucket, i.e., $PM\ node_3$.

D. Operations of Leader

1) **Point Query:** Algorithm 1 describes the query process in Leader. For a query key, the root node is searched to fetch the node ($node$) in the second layer (Line 1). Next, $node$ is searched to fetch the leaf node (seg) in the third layer (Line 2).

Algorithm 1 Query of Leader.

Input: the query key .

Output: the predicted search range.

```
1:  $node \leftarrow \text{SearchNode}(\text{root}, key)$ 
2:  $seg \leftarrow \text{SearchNode}(node, key)$ 
3:  $pos \leftarrow \lfloor seg.slope \times key + seg.intercept \rfloor$ 
4: return  $\{pos - error\_bound, pos + error\_bound\}$ 
5:
6: Function  $\text{SearchNode}(node, key)$ 
7:    $pos \leftarrow node.model.slope \times key + node.model.intercept$ 
8:    $type \leftarrow node.entries[pos].type$ 
9:    $child \leftarrow node.entries[pos].child$ 
10:  if  $key$  is in the range of  $child$  then
11:    if  $type$  is Bucket Pointer then
12:       $child \leftarrow child.locate(key)$ 
13:    end if
14:    return  $child$ 
15:  else
16:     $re\_child \leftarrow node.entries[pos].redirect\_child$ 
17:    if  $key$  is not in the range of  $re\_child$  then
18:       $key$  does not exist and the query is terminated
19:    end if
20:    return  $re\_child$ 
21:  end if
```

Finally, utilizing the linear model in seg , the position of the key is predicted, and the final search range can be obtained (Lines 3-4). The detailed search process in a model node is illustrated in the function SearchNode (Line 6). First, the model of the node predicts the position of the entry (Line 7). Then, the child indexed by the entry is fetched (Line 9). If the key falls in the range of the child and the child is a bucket, a binary search is performed within the bucket to select a model node for the subsequent search (Lines 10-13). If the key falls in the range of the child and the child is a model node, the model node is returned directly (Line 14). If the key does not fall in the range of the child, the redirect pointer stored in the predicted entry is used to fetch the redirect model node (Lines 15-16). If the key does not fall in the range of the original child and the redirect model node, it indicates that the key does not exist, and the search process is terminated (Lines 17-19). Otherwise, the model node is returned (Line 20).

2) *Node Construction*: Algorithm 2 describes the construction of PM nodes at internal layers in Leader. The algorithm extracts the minimum key of each node in $children$ to form a minimum key sequence (Line 1). The ordinary least squares regression is applied to train a linear regression model (Line 2). Subsequently, the algorithm computes the storage position for each key using the linear model (Line 4). All entry types are initially set to be null. If the type of the predicted entry is null, the entry stores the address of the model node. The corresponding entry type is marked as a node pointer (Lines 6-7). If multiple model nodes are mapped to the same position, the child indexed by the entry is updated to a bucket (Line 9), and all model nodes mapped to the bucket are stored in order (Line 10). After the above process, the type of an entry is null if it is not marked as a node pointer. Based on the redirect mechanism described in Section III-C, if the type of an entry

Algorithm 2 Construction of a PM node.

Input: $children$ (AM nodes or PM nodes) to be indexed by the current constructing node.

Output: the constructed model node containing a linear model M and entry information $entries$.

```
1:  $keys \leftarrow$  extract the minimal key of each node in  $children$ 
2:  $M \leftarrow \text{OLS}(keys)$ 
3: for each  $child$  in  $children$  do
4:    $pos \leftarrow \lfloor M.slope \times child.minkey + M.intercept \rfloor$ 
5:   if  $entries[pos].type$  is Null then
6:      $entries[pos].type \leftarrow$  Node Pointer
7:      $entries[pos].child \leftarrow child$ 
8:   else
9:      $entries[pos].type \leftarrow$  Bucket Pointer
10:     $entries[pos].child \leftarrow$  bucket containing children mapped to  $pos$ 
11:  end if
12: end for
13:  $last\_child \leftarrow null$ 
14: for each  $entry$  in  $entries$  do
15:   if  $entry.type$  is Node Pointer then
16:      $entry.redirect\_child \leftarrow last\_child$ 
17:      $last\_child \leftarrow entry.child$ 
18:   else if  $entry.type$  is Bucket Pointer then
19:      $entry.redirect\_child \leftarrow last\_child$ 
20:      $last\_child \leftarrow$  the last model node in the bucket
21:   else if  $entry.type$  is Null then
22:      $entry.child \leftarrow last\_child$ 
23:      $entry.type \leftarrow$  node pointer
24:   end if
25: end for
26: return  $\text{Node}(M, entries)$ 
```

is node pointer or bucket pointer, the redirect pointer at the entry points to the previous child model node (Lines 15-16 and Lines 18-19). The algorithm updates $last_child$ with the previous model node (Line 17 and Line 20). If the type of an entry is null, the entry stores the redirect pointer as the child pointer and updates its type as a node pointer (Lines 22-23). Finally, the constructed model node is returned (Line 26).

3) *Leader Construction*: Algorithm 3 describes the construction of Leader. First, piece-wise linear regression is employed to train AM nodes at the leaf layer (Line 1). To construct the second layer, Leader groups AM nodes into orderly disjoint groups in sets of $group_size$ (Line 2). Within each group, AM nodes are used to construct the upper-layer PM node (Lines 5-7). Once PM nodes in the second layer are all constructed, they are used to build the root node using the algorithm described in Algorithm 2 (Lines 9-10).

E. LeaderKV: Integrating DKTable and Leader into LSM-tree

LeaderKV integrates DKTable and Leader into LSM-tree. When DKTable completes the construction of the value area, keys in the newly generated DKTable are collected and placed in an asynchronous background learning queue. The queue trains Leader for the DKTable. After Leader is trained, it is persisted in an index file, which stores the parameters of AM nodes and PM nodes. When the index is loaded next time, PM nodes store pointers in the entries after child nodes are constructed. The index file also stores the metadata (e.g., the

Algorithm 3 Construction of Leader.**Input:** the sorted key sequence $keys$.**Output:** the root node of Leader

```

1:  $AMs \leftarrow \text{PLR}(keys)$ 
2:  $G \leftarrow \text{group } AMs \text{ in sets of } group\_size \text{ in order}$ 
3:  $PMs \leftarrow []$ 
4: for each  $group$  in  $G$  do
5:    $am\_children \leftarrow \text{AM nodes in } group$ 
6:    $PM \leftarrow \text{BuildNode}(am\_children)$ 
7:    $PMs.add(PM)$ 
8: end for
9:  $root \leftarrow \text{BuildNode}(PMs)$ 
10: return  $root$ 

```

number of key-vinfo pairs in a block of the DKTable), which is necessary for model prediction. During queries, searching for a KV pair follows the steps below.

- *Seek Table*: Identify DKTables that may contain the key.
- *Load FB/LF*: Load the filter block and learned index file.
- *Model Predict*: Leader predicts the position (pos) of the target key in the key area.
- *Seek FB*: Based on pos , query the corresponding filter block to identify the existence of the key. Since Bloom filters never return false negatives, the search within the DKTable is terminated if the filter indicates a negative.
- *Load Keys*: Read key-vinfo pairs starting from $(\max(pos-e, 0) \times (key_size + vinfo_size))$ th byte with a maximum size of $(2 \times e + 1) \times (key_size + vinfo_size)$ bytes in the key area. (e refers to the error bound)
- *Seek Key*: Perform a binary search within the loaded key-vinfo pairs to locate the target key.
- *Read Value*: If the key is found, read the value from the value area according to its vinfo.

F. Limitations

In our approach, LeaderKV requires the key size to be constant. Although the fixed key size ensures efficient retrieval through simple computation, it constrains the system's flexibility. Further research is needed to develop learned indexes that can handle variable key size. Another limitation of our approach comes from model training. LeaderKV needs to train a model for each DKTable, which consumes additional time. To mitigate the impact of model training on foreground processes, LeaderKV configures the training in the background. Experiments (see Section IV-H) show that the training time occupies 1.72% of LeaderKV's construction time, indicating that the training overhead is negligible.

IV. EXPERIMENTS**A. Experimental Setup**

Datasets. We use four real-world datasets from Search on Sorted Data Benchmark (SOSD) [32] and two synthetic datasets to evaluate LeaderKV. All datasets consist of 64 million KV pairs. The key size is 20 bytes, and the value size is 64 bytes. All keys and values are in string format. Below is the specific information of each dataset.

TABLE II: Workloads of YCSB. Workload-D uses the Latest distribution, and other workloads use Zipfian distribution.

Workload	Operations
A	50% reads, 50% updates
B	95% reads, 5% updates
C	100% reads
D	95% reads, 5% inserts
E	95% scans (scan length: 100), 5% inserts
F	50% reads, 50% read-modify-write

- *BOOK*. Book sale popularity collected from Amazon.
- *FB*. An upsampled version of a Facebook user ID dataset.
- *OSM*. Uniformly sampled locations as Google S2 CellIds.
- *WIKI*. Edit timestamps for Wikipedia articles.
- *LOGN*. This synthetic dataset is generated from a lognormal distribution with parameters $\mu=0$ and $\sigma=2$, multiplied by 10^9 and rounded down to the nearest integer.
- *UNI*. This synthetic dataset is generated from a uniform distribution of 0 to 10^{16} .

Workloads. We generate four workloads. Each workload consists of 10 million operations. The read-only workload contains 100% read operations. The read-heavy workload contains 90% read operations and 10% insert operations. The balanced workload contains 50% read operations and 50% insert operations. The write-heavy workload contains 10% read operations and 90% insert operations. The queries follow a Zipfian distribution. The parameter $zipf$ is set to 0.99 (the default setting in YCSB [10]). We also use YCSB real benchmark to evaluate LeaderKV. YCSB contains six types of workload, which are shown in Table II. In the experiments, BOOK is designated as the data source of YCSB. The workloads of YCSB are configured to load 50 million records and execute 10 million operations.

Baselines. We select representative schemes for comparison.

- *Bourbon* [28]. A learned-index-aided KV store that provides fast lookups by learning key distribution.
- *RocksDB* [4]. An embedded, high-performance, and persistent KV store for fast storage.
- *LevelDB* [2]. A fast KV store developed by Google that provides an ordered mapping from keys to values.

Metrics. We use the following metrics to evaluate LeaderKV and other schemes.

- *Throughput*. The average rate of operations per second.
- *P99 latency*. The latency at the 99th percentile of all operations.
- *Index size*. The size of learned indexes and index blocks.
- *Construction time*. The duration taken for loading data.

Environment and Parameters. We implement LeaderKV in C++. All source files are compiled with GCC 9.4.0. All experiments are performed on a server with an Intel(R) Core i7-11700K CPU (16 logical cores, 3.6 GHz, 640 KB L1 Cache, 4 MB L2 Cache, and 16 MB LLC), 32 GB 3200 MHz DDR4 main memory and 1 TB Samsung NVMe SSD, running 64-bit Ubuntu 20.04.6 LTS on an ext4 partition. For all schemes, the bits-per-key of the Bloom filter is set to 10. Unless otherwise

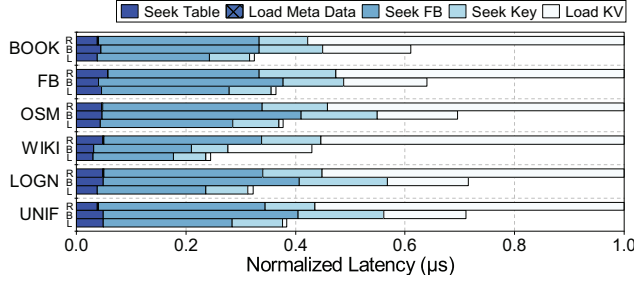


Fig. 7: Normalized latency breakdown of random lookup.

stated, the other parameters assume the default configuration of RocksDB. Specifically, for LeaderKV, we set the hyperparameters *error bound* (see Section III-C) and *group size* (see Section III-D3) to 16 and 64, respectively. This setting is explained in Section IV-I.

B. Latency Breakdown of Point Query

Figure 7 shows the random lookup latency breakdown for LeaderKV, Bourbon, and RocksDB on different datasets. On the y-axis, labels “R”, “B”, and “L” represent RocksDB, Bourbon, and LeaderKV, respectively. Load Meta Data refers to Load FB/LF in LeaderKV and Load IB/FB in other schemes. Seek Key refers to Model Predict / Seek Key in LeaderKV and Seek IB / Seek DB in other schemes. Load KV refers to Load Keys / Read Value in LeaderKV and Load DB in other schemes. From Figure 7, it can be observed that LeaderKV achieves the lowest random lookup latency by reducing the time of indexing (i.e., Seek Key) and data access (i.e., Load KV). Compared to Bourbon, LeaderKV reduces the time of indexing and data access by 32.96% and 94.30% on average, respectively. Compared to RocksDB, LeaderKV reduces the time of indexing and data access by 27.72% and 98.43% on average, respectively. In terms of indexing, LeaderKV uses Leader to accelerate key retrieval. The hybrid-model structure comprising precise models and approximate models contributes to performance improvement. In terms of data access, LeaderKV loads a maximum size of $(2 \times e + 1) \times (key_size + vinfo_size)$ bytes in the key area (e refers to the error bound), and loads *value_size* bytes in the value area in the target DKTable. This significantly reduces the cost of data access compared to Bourbon and RocksDB, which must load the entire or multiple data blocks during searching. From Figure 7, it can be observed that LeaderKV also reduces the time of Seek FB. This is attributed to fewer cache misses when querying filters. The filter size of LeaderKV decreases because fewer keys are stored in DKTable, which not only stores keys and values but also vinfos. Our experiments indicate that, on average, RocksDB, Bourbon, and LeaderKV store 590,307, 1,003,126, and 568,288 KV pairs in each file, respectively.

C. Throughput

1) *Read-Only / Read-Heavy Workloads*: Figure 8(a) shows the throughput of LeaderKV and other schemes under read-

only workloads. LeaderKV achieves the highest throughput of over 0.7 million operations per second (Mops/sec). Compared to Bourbon, RocksDB, and LevelDB, LeaderKV improves the throughput by 1.75 $\times$, 2.71 $\times$, and 3.68 $\times$ on average, respectively. Figure 8(b) shows the throughput of LeaderKV and other schemes under read-heavy workloads. LeaderKV demonstrates the highest throughput close to 0.6 Mops/sec. Compared to Bourbon, RocksDB, and LevelDB, LeaderKV improves the throughput by 1.57 $\times$, 2.64 $\times$, and 3.31 $\times$ on average, respectively. LeaderKV uses precise models to locate the target model efficiently. Besides, Bourbon, RocksDB, and LevelDB load the entire or multiple data blocks during queries. LeaderKV only loads a small account of data. This reduces irrelevant data access, contributing to performance improvement in read-intensive workloads.

2) *Balanced Workloads*: Figure 8(c) shows the throughput of LeaderKV and other schemes under balanced workloads. It can be observed that LeaderKV achieves the highest throughput of over 0.4 Mops/sec. Compared to Bourbon, RocksDB, and LevelDB, LeaderKV improves the throughput by 1.35 $\times$, 2.16 $\times$, and 2.86 $\times$ on average, respectively. LeaderKV utilizes DKTable and Leader to provide fast retrieval and reduced data access. Besides, LeaderKV does not introduce significant impacts in the critical write path since it decouples the training of Leader from DKTable construction.

3) *Write-Heavy Workloads*: Figure 8(d) shows the throughput of LeaderKV and other schemes under write-heavy workloads. LeaderKV achieves the best performance among all schemes, although the performance improvement is not as high as under the previous three workloads. The throughput of LeaderKV on different datasets is close to 0.4 Mops/sec. Compared to Bourbon, RocksDB, and LevelDB, LeaderKV achieves an average improvement of 1.27 $\times$, 1.6 $\times$, and 2.39 $\times$ in throughput, respectively. The performance improvement of LeaderKV under write-heavy workloads mainly comes from two aspects. First, as shown in Section IV-F, the index size of LeaderKV is generally smaller than that of Bourbon, RocksDB, and LevelDB. The reduced amount of data flushed to disk contributes to the decreased latency. Second, unlike SSTable, which needs to write the index block synchronously after all data blocks are flushed to disk, DKTable decouples the learned index. The learned index is persisted on disk asynchronously after the value area of DKTable is constructed, thus improving the write throughput of LeaderKV.

D. P99 Latency

1) *Read-Only / Read-Heavy Workloads*: Figure 9(a) shows the P99 latency of LeaderKV and other schemes under read-only workloads. It can be observed that LeaderKV exhibits the lowest P99 tail latency of about 2 μ s. Compared to Bourbon, RocksDB, and LevelDB, the P99 tail latency of LeaderKV is reduced by 39.99%, 66.87%, and 81.90% on average, respectively. Fig 9(b) shows the P99 latency of Leader and other schemes under read-heavy workloads. LeaderKV exhibits the lowest P99 latency of 4 μ s. Compared to Bourbon, RocksDB, and LevelDB, the P99 latency of LeaderKV is

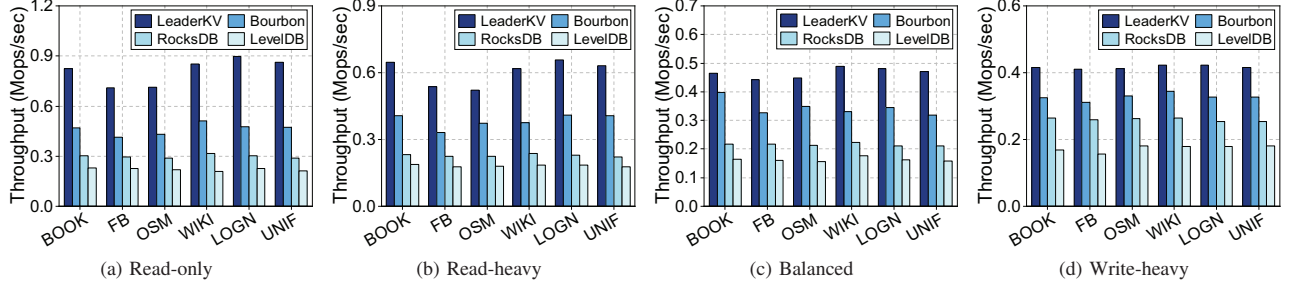


Fig. 8: Throughput of LeaderKV and baselines.

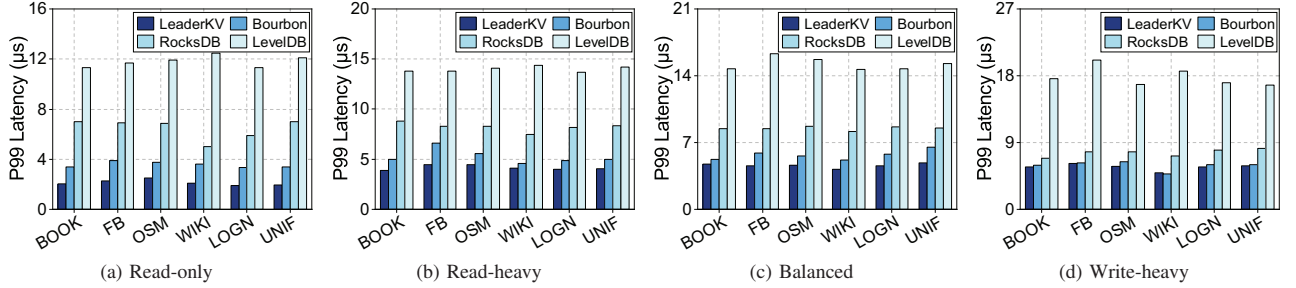


Fig. 9: P99 latency of LeaderKV and baselines.

reduced by 20.60%, 49.33%, and 70.17% on average, respectively. LeaderKV uses the hybrid-model learned index to locate the range that may contain the target key. This decreases the operation latency in the worst-case scenario.

2) *Balanced Workloads*: Figure 9(c) shows the P99 latency of LeaderKV and other schemes under balanced workloads. LeaderKV exhibits the lowest P99 latency of about 4.5 μ s. Compared to Bourbon, RocksDB, and LevelDB, the P99 latency of LeaderKV is reduced by 19.35%, 43.10%, and 68.28% on average, respectively. LeaderKV leverages Leader to accelerate data retrieval and decouples training from the critical path to reduce the impact on writing.

3) *Write-Heavy Workloads*: Figure 9(d) shows the P99 latency of LeaderKV and other schemes under write-heavy workloads. LeaderKV achieves the lowest P99 latency on most datasets, except for a slightly higher P99 latency compared to Bourbon on WIKI dataset. The average P99 latency of LeaderKV is about 5.7 μ s. Compared to Bourbon, RocksDB, and LevelDB, LeaderKV reduces the P99 tail latency by 3.78%, 13.52%, and 63.01% on average, respectively. Compared to RocksDB and LevelDB, LeaderKV decouples the index from DKTable's construction, which reduces the impact on DKTable construction. Bourbon stores values in the external files, while LeaderKV reserves values in DKTables. Therefore, the number of KV pairs in a storage file in LeaderKV is less than Bourbon. This may trigger more compaction if the amount of the loaded KV pairs is the same. However, Bourbon needs to execute garbage collection (GC) periodically, which results in serious performance degradation during GC.

E. Real Macrobenchmark

Figure 10 shows the throughput of LeaderKV and other schemes under six YCSB workloads. It can be observed that LeaderKV achieves the highest throughput across six workloads. Among all workloads, YCSB-B, YCSB-C, and YCSB-D are read-intensive workloads. The throughput of LeaderKV is 0.40 Mops/sec, 0.51 Mops/sec, and 0.81 Mops/sec under YCSB-B, YCSB-C, and YCSB-D workloads, respectively. Compared to Bourbon, RocksDB, and LevelDB, the throughput of LeaderKV achieves an average improvement of 1.35 $\times$, 2.19 $\times$, and 2.56 $\times$, respectively. LeaderKV utilizes Leader and DKTable to provide efficient and reduced data access, improving the throughput under read-intensive workloads.

YCSB-E is a scan-intensive workload. The throughput of LeaderKV is 0.1 Mops/sec under YCSB-E workload. Compared to Bourbon, RocksDB, and LevelDB, the throughput of LeaderKV achieves an average improvement of 9.80 $\times$, 2.61 $\times$, and 4.36 $\times$, respectively. The values in Bourbon scatter in external files, which significantly reduces the spatial locality. Therefore, the range query performance of Bourbon suffers a severe loss. LeaderKV, RocksDB, and LevelDB exhibit better spatial locality of KV pairs, leading to higher throughput for range queries. Besides, LeaderKV employs learned indexes to further accelerate queries.

YCSB-A and YCSB-F are balanced workloads. The throughput of LeaderKV is 0.26 Mops/sec and 0.23 Mops/sec under YCSB-A and YCSB-F workloads, respectively. Compared to Bourbon, RocksDB, and LevelDB, the throughput of LeaderKV achieves an average improvement of 1.33 $\times$, 2.03 $\times$, and 2.52 $\times$, respectively. LeaderKV utilizes the collaboration

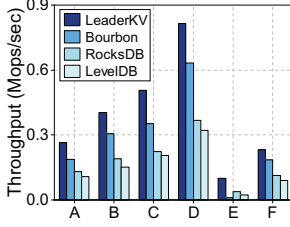


Fig. 10: Throughput of LeaderKV and baselines under YCSB workloads.

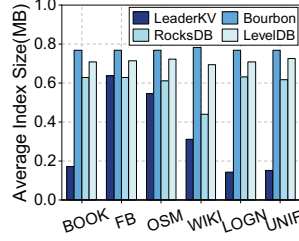


Fig. 11: Average index size of LeaderKV and baselines on six datasets.

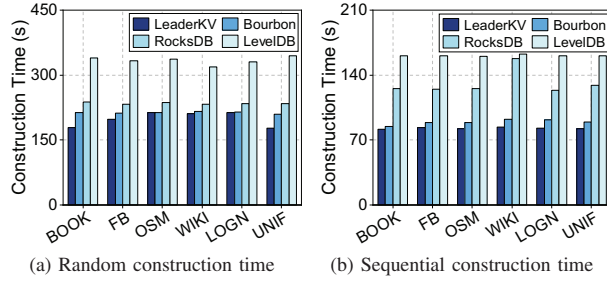


Fig. 12: Construction time of LeaderKV and baselines.

of Leader and DKTables to optimize the read performance. Besides, the decoupled design of DKTable and Leader reduces the impact on the critical path. These contribute to performance improvement under balanced workloads.

F. Storage Footprint

Figure 11 shows the average index size of LeaderKV and other schemes. This metric refers to the storage space consumed by the index in each data file. It can be observed that LeaderKV achieves the smallest index size on most datasets, except for similar index size compared to RocksDB on FB dataset. The average index size of LeaderKV is 0.33 MB. Compared to Bourbon, RocksDB, and LevelDB, LeaderKV reduces the index size by 57.53%, 44.81%, and 54.06%, respectively. LeaderKV utilizes Leader to replace the index block in LSM-tree. In Leader, AM nodes only store the parameters, and PM nodes store the parameters and navigation data. Typically, this reduces the index space consumption to keep indexes. The key distribution in FB dataset is more complicated than other datasets. More nodes are constructed in Leader to fit the CDF, thus increasing the index size.

G. Construction Time

Figure 12(a) shows construction time for LeaderKV and other schemes when data is loaded in random order. LeaderKV achieves the lowest construction time on most datasets, except for similar time compared to Bourbon on OSM dataset. Among all datasets, LeaderKV achieves an average construction time of 198.37 s. Compared to Bourbon, RocksDB, and LevelDB, the construction time of LeaderKV is reduced by 6.95%, 15.49%, and 40.58% on average, respectively.

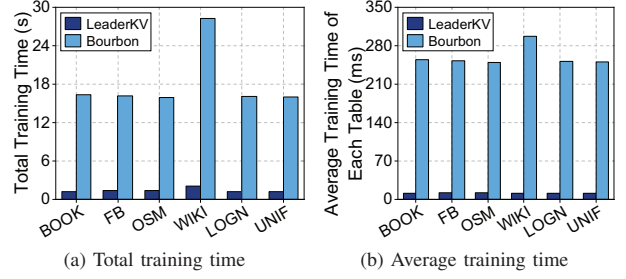


Fig. 13: Model training time of LeaderKV and Bourbon.

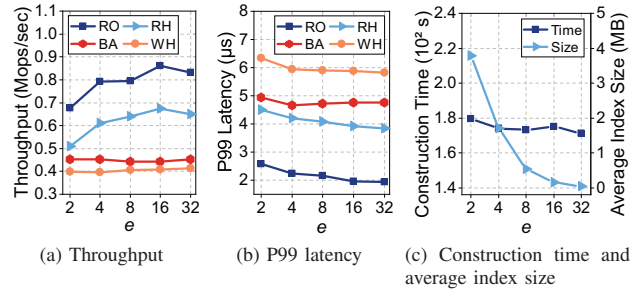


Fig. 14: Impact of *error bound* (e) on LeaderKV.

Figure 12(b) shows the construction time for LeaderKV and other schemes when data is loaded in sequential order. The construction time of LeaderKV is slightly lower than that of Bourbon and is the lowest among all schemes. Among all datasets, LeaderKV achieves an average construction time of 82.72 s. Compared to Bourbon, RocksDB, and LevelDB, the construction time of LeaderKV is reduced by 7.44%, 36.88%, and 48.63% on average, respectively. When loading data in random order, major compaction is triggered in all schemes. When loading data in sequential order, only minor compaction is triggered in all schemes. Data files in upper levels are simply moved to lower levels. Therefore, the sequential construction time is less than the random construction time for all schemes. LeaderKV decouples the training and flushing of the index from DKTable construction. The reduced index size and the decoupled design of DKTable and Leader decrease the construction time of LeaderKV.

H. Model Training Time

Figure 13(a) shows the total training time of LeaderKV and Bourbon. LeaderKV requires 1.42 s, while Bourbon requires 18.1 s for model training. Figure 13(b) shows the training time for each data file in LeaderKV and Bourbon. LeaderKV spends 11.59 ms, while Bourbon spends 259.22 ms on average. LeaderKV trains models in the background to minimize the impact on foreground processes. It imposes a looser constraint for error bound in the hybrid-model learned index, thus reducing the model training time.

I. Impact of Hyper-parameters

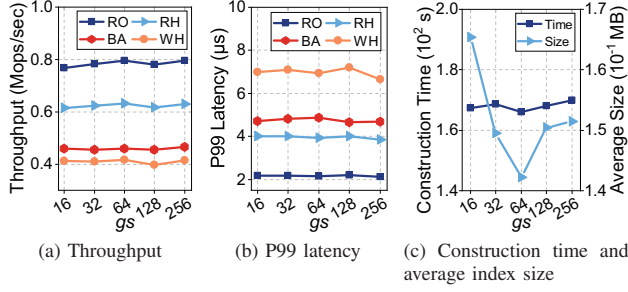


Fig. 15: Impact of *group size* (*gs*) on LeaderKV.

1) *Impact of error bound*: The *error bound* is a parameter defined in AM nodes in Leader (see Section III-C). Figure 14(a) shows the throughput of LeaderKV under different workloads as *error bound* changes. Legends “RO”, “RH”, “BA”, and “WH” represent read-only, read-heavy, balance, and write-heavy, respectively. Under read-only and read-heavy workloads, as *error bound* increases from 2 to 32, the throughput first increases and then decreases. When *error bound* equals 16, the throughput of LeaderKV reaches a maximum of 0.86 Mops/sec and 0.67 Mops/sec under read-only and read-heavy workloads, respectively. As *error bound* increases from 2 to 16, the number of nodes constructed at the leaf layer and the second layer decreases. Therefore, the size of the root node also reduces. These factors contribute to fewer cache misses and improve performance. However, as *error bound* grows, the cost to locate the key at the predicted range increases, resulting in throughput degradation. For balanced and write-heavy workloads, it can be observed that *error bound* does not significantly impact the throughput of LeaderKV. The throughput of LeaderKV is about 0.45 Mops/sec and 0.41 Mops/sec as *error bound* changes under balanced and write-heavy workloads, respectively. The *error bound* mainly influences the number of leaf nodes. The increased *error bound* leads to fewer leaf nodes. LeaderKV decouples the training of learned indexes from DKTable construction, which reduces the impact of *error bound* on writing. Besides, the target data can be robustly located via the hybrid-model learned index.

Figure 14(b) shows the P99 latency of LeaderKV under different workloads as *error bound* changes. As *error bound* increases from 2 to 32, the P99 latency of LeaderKV slightly decreases from 6.34 μs to 5.82 μs under write-heavy workloads. The increased *error bound* leads to decreased computation, which reduces the competition for computing resources and the impact on the write path in LeaderKV in the worst-case scenarios. Under balanced workloads, as *error bound* changes, the P99 latency of LeaderKV remains stable at around 4.76 μs. Under read-heavy workloads, as *error bound* increases, the P99 latency decreases from 4.50 μs to 3.84 μs. Under read-only workloads, as *error bound* increases, the P99 latency decreases from 2.59 μs to 1.93 μs. As *error bound* increases, the number of nodes at the leaf layer and the second layer decreases, leading to fewer children indexed by the root

node. In the worst case, this reduces the number of conflicts at the root, leading to a slight decrease in P99 latency.

Figure 14(c) shows the construction time and the index size of LeaderKV as *error bound* changes. As *error bound* increases from 2 to 32, the construction time fluctuates at around 174.64 s because the training of the learned index is decoupled from the critical write path, which minimizes the influence on construction. As *error bound* increases from 2 to 32, the size of Leader decreases from 3.78 MB to 0.04 MB because the number of nodes in Leader significantly decreases.

From Figure 14, it can be seen that when *error bound* is 16, Leader achieves the highest throughput without bringing significant influence to the construction time and index size. Therefore, LeaderKV sets *error bound* to 16.

2) *Impact of group size*: The *group size* is a parameter defined in the construction of Leader (see Section III-D3). It impacts the number of nodes in each group during construction. Figure 15(a) shows the impacts of *group size* on the throughput of LeaderKV under different workloads when *error bound* is fixed to 16. As *group size* increases from 16 to 256, the impact on throughput is insignificant. The throughput under read-only, read-heavy, balanced, and write-heavy workloads is around 0.78 Mops/sec, 0.62 Mops/sec, 0.46 Mops/sec, and 0.41 Mops/sec, respectively. Figure 15(b) shows the influence of *group size* on P99 latency of LeaderKV under different workloads when *error bound* is fixed to 16. The P99 latency of LeaderKV is also not sensitive to the changes of *group size*. As *group size* increases from 16 to 256, the P99 latency on read-only, read-heavy, balanced, and write-heavy workloads is around 2.17 μs, 3.96 μs, 4.76 μs, and 6.98 μs.

Figure 15(c) shows the impacts of *group size* on construction time and index size of LeaderKV when *error bound* is fixed to 16. As *group size* increases from 16 to 256, the construction time fluctuates at around 167.99 s. The training of learned indexes is decoupled from the critical write path to minimize the impact on construction. The index size first decreases and then increases. As *group size* grows from 16 to 64, the index size decreases from 0.165 MB to 0.142 MB. As *group size* grows to 256, the index size increases to 0.152 MB. The size of Leader is affected by the number of nodes and the average node size. As *group size* increases, the number of nodes in Leader decreases, while the average node size increases with the growing size of entries. When *group size* is 64, the index size of Leader is the smallest.

From Figure 15, it can be seen that when *group size* is 64, LeaderKV achieves the smallest index size without compromising throughput, P99 latency, and construction time. Therefore, LeaderKV sets *group size* to 64.

V. RELATED WORK

A. Read Optimizations towards LSM-Tree-Based KV Stores

1) *Cache Optimization*: AC-Key [17] proposes an adaptive caching algorithm to adjust the cache size according to the workload. LSbM-tree [19] adds a buffer to minimize cache invalidations caused by compactions. CacheKV [33] is built atop persistent CPU caches. Although caching accelerates lookups

of recently accessed data, it consumes non-negligible storage as the cache size increases. LeaderKV leverages a lightweight learned index to improve read performance without bringing significant storage overhead in LSM-tree.

2) *Filter Optimization*: Several works utilize filters to skip unnecessary reading since the filters do not return a false negative [20]–[23]. However, the filters suffer from high false-positive rates and excessive memory usage. LeaderKV gets rid of these by designing a lightweight learned index to locate the target data efficiently.

3) *Index Optimization*: KVell [26] utilizes NVMe SSDs at maximum bandwidth to boost index performance. FASTER [24] proposes a highly cache-optimized concurrent hash index. SLM-DB [25] keeps a B+-Tree in persistent memory to provide high throughput. These indexes consume significant space and leave room for performance improvement. Bourbon [28] analyzes the feasibility of applying learned indexes to LSM-tree. Google integrates learned indexes into BigTable [29]. TridentKV [1] designs learned index blocks in SSTables, uses String Number Base Coding (SNBC) to transform strings to numbers for model training and prediction, and employs Storage Performance Development Kit (SPDK) [34] to accelerate asynchronous reading. LeaderKV designs a hybrid-model learned index and overcomes the overhead of disk access in Bourbon and TridentKV via the collaboration of DKTable and Leader. The SNBC and SPDK used in TridentKV can be applied to LeaderKV. To support queries for non-primary attributes, LevelDB++ [35], SineKV [36], and Lark [37] are proposed. Different from them, our work focuses on the read optimization for primary keys in LSM-tree.

B. Learned Index

The learned index proposed by Tim et al. [27] inspires a series of works. ALEX [38], PGM-Index [39], FITing-Tree [40], and LIPP [41] are proposed to support dynamic workloads. LISA [42], Flood [43], and Tsunami [44] are designed to support multi-dimensional learned indexes. XIndex [45] and FINEdex [46] investigate the concurrency of learned indexes. SIndex [47] explores string support for learned indexes. They are different from our scope. Our work focuses on designing a learned index for efficient data retrieval in LSM-tree. The effect of learned indexes, i.e., reading fewer index blocks, is exploited to reduce the cost of disk access. NFL [48] uses normalizing flow to improve the approximation quality of learned indexes, which can be combined with our solution. APEX [49] and PLIN [50] are designed on persistent memory. Some works integrate learned indexes with hardware to improve the hardware performance. ROLEX [51] integrates learned indexes into disaggregated memory systems to process data requests via one-sided Remote Direct Memory Access (RDMA) operations. LeaFTL [52] integrates learned indexes into the flash translation layer (FTL) to reduce the size of mapping tables. Different from them, LeaderKV designs a hybrid-model learned index for LSM-tree on the software level. It would be interesting to leverage the learned-index-enhanced hardware to improve the performance of our work.

VI. CONCLUSION

This paper presents LeaderKV, a read-optimized KV store enhanced by learned indexes and decoupled KV tables. DKTable decouples keys and values into distinct areas and fits well with learned indexes to reduce disk access. A hybrid-model learned index called Leader is designed to improve the query efficiency within DKTable. We implement LeaderKV by integrating DKTable and Leader into LSM-tree. Experimental results demonstrate the effectiveness of LeaderKV.

In the future, we plan to utilize the high internal bandwidth of computational storage to improve LeaderKV's performance. We also plan to offload LeaderKV's training and inference to GPU and FPGA to boost the training and query performance.

ACKNOWLEDGMENT

This work was supported in part by NSFC (62122056, 62102263, U23B2040, and U2001212), in part by Guangdong Basic and Applied Basic Research Foundation (2020B1515120028, 2022A1515010180), in part by Shenzhen Science and Technology Program (RCJC20221008092725019, JCYJ20210324094402008, JCYJ20210324094208024, and 20220810144025001), in part by Tencent "Rhinoceros Birds" - Scientific Research Foundation for Young Teachers of Shenzhen University, in part by CCF-Tencent Rhino-Bird Open Research Fund (RAGR20230126), in part by National Key R&D program of China (2021YFB3301500), in part by Shenzhen University Research Instrument Development and Cultivation (2023YQ017), in part by the Foundation of Shenzhen Grant number (20220810142731001), and in part by Guangdong Provincial Key Laboratory of Popular High Performance Computers and Shenzhen Key Laboratory of Service Computing and Applications. Corresponding authors: Chenlin Ma and Jianbin Qin.

REFERENCES

- [1] K. Lu, N. Zhao, J. Wan, C. Fei, W. Zhao, and T. Deng, "TridentKV: A Read-Optimized LSM-Tree Based KV Store via Adaptive Indexing and Space-Efficient Partitioning," *IEEE Transactions on Parallel and Distributed Systems (TPDS)*, vol. 33, no. 8, pp. 1953–1966, 2022.
- [2] Google, "LevelDB," 2023. [Online]. Available: <https://code.google.com/p/leveldb>
- [3] F. Chang, J. Dean, S. Ghemawat, W. C. Hsieh, D. A. Wallach, M. Burrows, T. Chandra, A. Fikes, and R. E. Gruber, "Bigtable: A Distributed Storage System for Structured Data," *ACM Transactions on Computer Systems (TOCS)*, vol. 26, no. 2, pp. 1–26, 2008.
- [4] Meta, "RocksDB," 2023. [Online]. Available: <https://rocksdb.org>
- [5] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Voshall, and W. Vogels, "Dynamo: Amazon's Highly Available Key-Value Store," in *Symposium on Operating Systems Principles (SOSP)*, 2007, pp. 205–220.
- [6] Apache, "Cassandra," 2023. [Online]. Available: <http://cassandra.apache.org>
- [7] —, "AsterixDB," 2023. [Online]. Available: <https://asterixdb.apache.org>
- [8] P. E. O'Neil, E. Cheng, D. Gawlick, and E. J. O'Neil, "The Log-Structured Merge-Tree (LSM-Tree)," *Acta Informatica*, vol. 33, no. 4, pp. 351–385, 1996.
- [9] X. Wu, Y. Xu, Z. Shao, and S. Jiang, "LSM-trie: An LSM-tree-based Ultra-Large Key-Value Store for Small Data Items," in *Annual Technical Conference (ATC)*, 2015, pp. 71–82.
- [10] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, "Benchmarking Cloud Serving Systems with YCSB," in *Symposium on Cloud Computing (SoCC)*, 2010, pp. 143–154.

- [11] T. G. Armstrong, V. Ponnemanti, D. Borthakur, and M. Callaghan, "LinkBench: A Database Benchmark Based on the Facebook Social Graph," in *International Conference on Management of Data (SIGMOD)*, 2013, pp. 1185–1196.
- [12] Meta, "ZippyDB," 2023. [Online]. Available: <https://engineering.fb.com/2021/08/06/core-infra/zippydb>
- [13] Z. Cao, S. Dong, S. Vemuri, and D. H. Du, "Characterizing, Modeling, and Benchmarking RocksDB Key-Value Workloads at Facebook," in *Conference on File and Storage Technologies (FAST)*, 2020, pp. 209–223.
- [14] C. Ma, H. Yang, S. Wu, Y. Wang, and R. Mao, "Tidal-Tree-Mem: Toward Read-Intensive Key-Value Stores with Tidal Structure Based on LSM-Tree," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, vol. 42, no. 2, pp. 423–436, 2023.
- [15] S. Sarkar, N. Dayan, and M. Athanassoulis, "The LSM Design Space and its Read Optimizations," in *International Conference on Data Engineering (ICDE)*, 2023, pp. 3578–3584.
- [16] C. Luo and M. J. Carey, "LSM-Based Storage Techniques: A Survey," *The VLDB Journal (VLDBJ)*, vol. 29, no. 1, pp. 393–418, 2019.
- [17] F. Wu, M.-H. Yang, B. Zhang, and D. H. Du, "AC-Key: Adaptive Caching for LSM-based Key-Value Stores," in *Annual Technical Conference (ATC)*, 2020, pp. 603–615.
- [18] D. Siying, C. Mark, G. Leonidas, B. Dhruba, S. Tony, and S. Michael, "Optimizing Space Amplification in RocksDB," in *Biennial Conference on Innovative Data Systems Research (CIDR)*, 2017, pp. 1–9.
- [19] D. Teng, L. Guo, R. Lee, F. Chen, S. Ma, Y. Zhang, and X. Zhang, "LSbM-tree: Re-Enabling Buffer Caching in Data Management for Mixed Reads and Writes," in *International Conference on Distributed Computing Systems (ICDCS)*, 2017, pp. 68–79.
- [20] Y. Li, C. Tian, F. Guo, C. Li, and Y. Xu, "ElasticBF: Elastic Bloom Filter with Hotness Awareness for Boosting Read Performance in Large Key-Value Stores," in *Annual Technical Conference (ATC)*, 2019, pp. 739–752.
- [21] H. Zhang, H. Lim, V. Leis, D. G. Andersen, M. Kaminsky, K. Keeton, and A. Pavlo, "SuRF: Practical Range Query Filtering with Fast Succinct Tries," in *International Conference on Management of Data (SIGMOD)*, 2018, pp. 323–336.
- [22] W. Zhong, C. Chen, X. Wu, and S. Jiang, "REMIX: Efficient Range Query for LSM-trees," in *Conference on File and Storage Technologies (FAST)*, 2021, pp. 51–64.
- [23] N. Dayan, M. Athanassoulis, and S. Idreos, "Monkey: Optimal Navigable Key-Value Store," in *International Conference on Management of Data (SIGMOD)*, 2017, pp. 79–94.
- [24] B. Chandramouli, G. Prasaad, D. Kossmann, J. Levandoski, J. Hunter, and M. Barnett, "FASTER: A Concurrent Key-Value Store with In-Place Updates," in *International Conference on Management of Data (SIGMOD)*, 2018, pp. 275–290.
- [25] O. Kaiyakhmet, S. Lee, B. Nam, S. H. Noh, and Y. ri Choi, "SLM-DB: Single-Level Key-Value Store with Persistent Memory," in *Conference on File and Storage Technologies (FAST)*, 2019, pp. 191–205.
- [26] B. Lepers, O. Balmau, K. Gupta, and W. Zwaenepoel, "KVell: The Design and Implementation of a Fast Persistent Key-Value Store," in *Symposium on Operating Systems Principles (SOSP)*, 2019, pp. 447–461.
- [27] T. Kraska, A. Beutel, E. H. Chi, J. Dean, and N. Polyzotis, "The Case for Learned Index Structures," in *International Conference on Management of Data (SIGMOD)*, 2018, pp. 489–504.
- [28] Y. Dai, Y. Xu, A. Ganesan, R. Alagappan, B. Kroth, A. Arpac-Dusseau, and R. Arpac-Dusseau, "From WiscKey to Bourbon: A Learned Index for Log-Structured Merge Trees," in *Symposium on Operating Systems Design and Implementation (OSDI)*, 2020, pp. 155–171.
- [29] H. Abu-Libdeh, D. Altunbiken, A. Beutel, E. H. Chi, L. Doshi, T. Kraska, L. Xiao Zhou, A. Ly, and C. Olston, "Learned Indexes for a Google-scale Disk-based Database," in *Proceedings of the Workshop on ML for Systems at NeurIPS (NIPS)*, 2020, pp. 1–5.
- [30] Q. Xie, C. Pang, X. Zhou, X. Zhang, and K. Deng, "Maximum Error-Bounded Piecewise Linear Representation for Online Stream Approximation," *The VLDB journal (VLDBJ)*, vol. 23, no. 6, pp. 915–937, 2014.
- [31] Wikipedia, "Ordinary Least Squares," 2023. [Online]. Available: https://en.wikipedia.org/wiki/Ordinary_least_squares
- [32] A. Kipf, R. Marcus, A. van Renen, M. Stoian, A. Kemper, T. Kraska, and T. Neumann, "SOSD: A Benchmark for Learned Indexes," *arXiv preprint arXiv:1911.13014*, pp. 1–5, 2019.
- [33] Y. Zhong, Z. Shen, Z. Yu, and J. Shu, "Redesigning High-Performance LSM-based Key-Value Stores with Persistent CPU Caches," in *International Conference on Data Engineering (ICDE)*, 2023, pp. 1098–1111.
- [34] Intel, "Storage Performance Development Kit (SPDK)," 2023. [Online]. Available: <https://spdk.io>
- [35] M. A. Qader, S. Cheng, and V. Hristidis, "A Comparative Study of Secondary Indexing Techniques in LSM-Based NoSQL Databases," in *International Conference on Management of Data (SIGMOD)*, 2018, pp. 551–566.
- [36] F. Li, Y. Lu, Z. Yang, and J. Shu, "SineKV: Decoupled Secondary Indexing for LSM-based Key-Value Stores," in *International Conference on Distributed Computing Systems (ICDCS)*, 2020, pp. 1112–1122.
- [37] J. Yuan, H. Liu, S. Wu, Y. Lin, T. Wang, C. Ma, R. Mao, and Y. Wang, "Work-in-Progress: Lark: A Learned Secondary Index Toward LSM-tree for Resource-Constrained Embedded Storage Systems," in *International Conference on Hardware/Software Codesign and System Synthesis (CODES+ISSS)*, 2022, pp. 11–12.
- [38] J. Ding, U. F. Minhas, J. Yu, C. Wang, J. Do, Y. Li, H. Zhang, B. Chandramouli, J. Gehrke, D. Kossmann, D. Lomet, and T. Kraska, "ALEX: An Updatable Adaptive Learned Index," in *International Conference on Management of Data (SIGMOD)*, 2020, pp. 969–984.
- [39] P. Ferragina and G. Vinciguerra, "The PGM-Index: A Fully-Dynamic Compressed Learned Index with Provable Worst-Case Bounds," *Proceedings of the VLDB Endowment (PVLDB)*, vol. 13, no. 8, pp. 1162–1175, 2020.
- [40] A. Galakatos, M. Markovitch, C. Binnig, R. Fonseca, and T. Kraska, "FITing-Tree: A Data-Aware Index Structure," in *International Conference on Management of Data (SIGMOD)*, 2019, pp. 1189–1206.
- [41] J. Wu, Y. Zhang, S. Chen, J. Wang, Y. Chen, and C. Xing, "Updatable Learned Index with Precise Positions," *Proceedings of the VLDB Endowment (PVLDB)*, vol. 14, no. 8, pp. 1276–1288, 2021.
- [42] P. Li, H. Lu, Q. Zheng, L. Yang, and G. Pan, "LISA: A Learned Index Structure for Spatial Data," in *International Conference on Management of Data (SIGMOD)*, 2020, pp. 2119–2133.
- [43] V. Nathan, J. Ding, M. Alizadeh, and T. Kraska, "Learning Multi-Dimensional Indexes," in *International Conference on Management of Data (SIGMOD)*, 2020, pp. 985–1000.
- [44] J. Ding, V. Nathan, M. Alizadeh, and T. Kraska, "Tsunami: A Learned Multi-Dimensional Index for Correlated Data and Skewed Workloads," *Proceedings of the VLDB Endowment (PVLDB)*, vol. 14, no. 2, pp. 74–86, 2020.
- [45] C. Tang, Y. Wang, Z. Dong, G. Hu, Z. Wang, M. Wang, and H. Chen, "XIndex: A Scalable Learned Index for Multicore Data Storage," in *Symposium on Principles and Practice of Parallel Programming (PPoPP)*, 2020, pp. 308–320.
- [46] P. Li, Y. Hua, J. Jia, and P. Zuo, "FINEdex: A Fine-Grained Learned Index Scheme for Scalable and Concurrent Memory Systems," *Proceedings of the VLDB Endowment (PVLDB)*, vol. 15, no. 2, pp. 321–334, 2021.
- [47] Y. Wang, C. Tang, Z. Wang, and H. Chen, "SIndex: A Scalable Learned Index for String Keys," in *Asia-Pacific Workshop on Systems (APSys)*, 2020, pp. 17–24.
- [48] S. Wu, Y. Cui, J. Yu, X. Sun, T.-W. Kuo, and C. J. Xue, "NFL: Robust Learned Index via Distribution Transformation," *Proceedings of the VLDB Endowment (PVLDB)*, vol. 15, no. 10, pp. 2188–2200, 2022.
- [49] B. Lu, J. Ding, E. Lo, U. F. Minhas, and T. Wang, "APEX: A High-Performance Learned Index on Persistent Memory," *Proceedings of the VLDB Endowment (PVLDB)*, vol. 15, no. 3, pp. 597–610, 2021.
- [50] Z. Zhang, Z. Chu, P. Jin, Y. Luo, X. Xie, S. Wan, Y. Luo, X. Wu, P. Zou, C. Zheng, G. Wu, and A. Rudoff, "PLIN: A Persistent Learned Index for Non-Volatile Memory with High Performance and Instant Recovery," *Proceedings of the VLDB Endowment (PVLDB)*, vol. 16, no. 2, pp. 243–255, 2022.
- [51] P. Li, Y. Hua, P. Zuo, Z. Chen, and J. Sheng, "ROLEX: A Scalable RDMA-oriented Learned Key-Value Store for Disaggregated Memory Systems," in *Conference on File and Storage Technologies (FAST)*, 2023, pp. 99–114.
- [52] J. Sun, S. Li, Y. Sun, C. Sun, D. Vucinic, and J. Huang, "LeaFTL: A Learning-Based Flash Translation Layer for Solid-State Drives," in *International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2023, pp. 442–456.